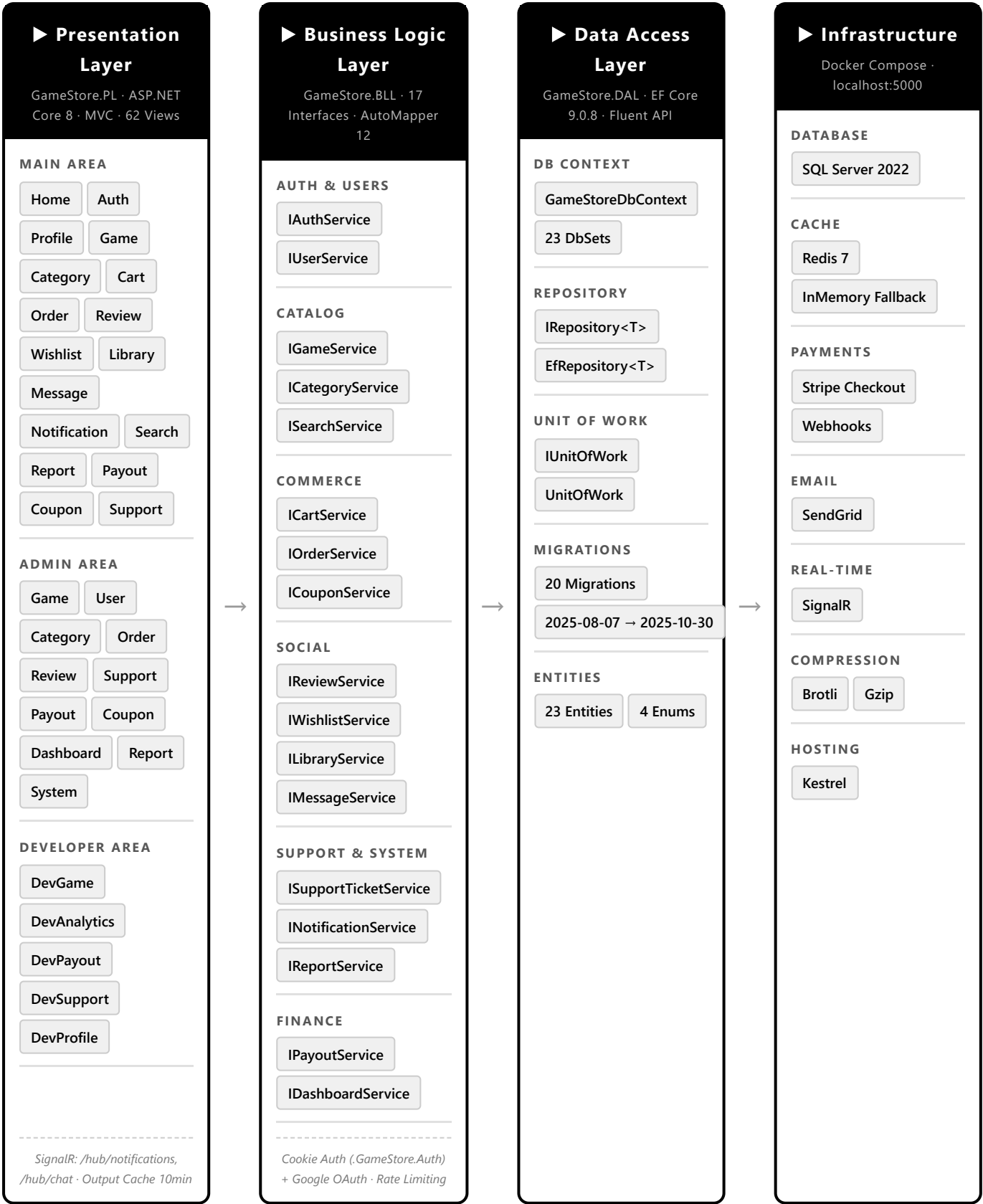


Implementation - GameStore Platform

1. Source Code Architecture

The GameStore platform follows a three-layer architecture separating presentation, business logic, and data access concerns. The solution contains four projects: **GameStore.PL** (ASP.NET Core MVC web application), **GameStore.BLL** (business logic services), **GameStore.DAL** (data access with EF Core + Repository pattern), and **GameStore.Tests** (xUnit unit tests). The diagram below illustrates the full project structure including all 33 controllers across three areas, the BLL service layer, and the DAL components.



2.1 Authentication Flow

The platform uses **Cookie-based authentication** (not JWT) with ASP.NET Core's built-in `CookieAuthenticationDefaults` scheme. Users authenticate via email/password or Google OAuth 2.0. Sessions are stored server-side with an 8-hour sliding expiration. The authentication pipeline includes a custom email verification middleware that blocks unverified users from accessing non-auth routes.

```
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options => {
        options.LoginPath          = "/Auth/Login";
        options.LogoutPath         = "/Auth/Logout";
        options.AccessDeniedPath   = "/Home/Index";
        options.Cookie.Name        = ".GameStore.Auth";
        options.Cookie.HttpOnly    = true;
        options.Cookie.SameSite    = SameSiteMode.Lax;
        options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
        options.ExpireTimeSpan     = TimeSpan.FromHours(8);
        options.SlidingExpiration  = true;
    })
    .AddGoogle(options => {
        options.ClientId           = builder.Configuration["Google:ClientId"] ?? "";
        options.ClientSecret       = builder.Configuration["Google:ClientSecret"] ?? "";
        options.CallbackPath       = "/auth/google-callback";
        options.SignInScheme       = CookieAuthenticationDefaults.AuthenticationScheme;
        options.SaveTokens         = true;
        options.Scope.Add("profile");
        options.Scope.Add("email");
    });
```

Password hashing uses BCrypt.Net-Next 4.0.3. Email verification tokens are stored in a dedicated `EmailVerificationTokens` table with expiry. A `TokenCleanupService` background service periodically purges expired tokens.

2.2 Shopping Cart Flow

The shopping cart is stored in the database via the `Carts` and `CartItems` tables, linked to the authenticated user. The `CartController` provides `Add`, `Remove`, and `GetCount` endpoints. Free games bypass Stripe and are processed via `CompleteFreeCheckoutAsync` which directly adds the game to the user's library. Paid games proceed through Stripe Checkout Sessions.

2.3 Stripe Payment Integration

Payments are processed through Stripe's Checkout API (Stripe.net 52.1.0). The `CartController.Index` creates a Stripe CheckoutSession with line items representing cart contents plus a success/cancel URL pair. The `StripeWebhookController` listens for `checkout.session.completed` events to finalize orders. A per-order commission percentage (stored in `CommissionPercent` on `OrderItems`) tracks the platform's revenue share for developer payouts.

```
var options = new SessionCreateOptions {
    PaymentMethodTypes = new List<string> { "card" },
    LineItems = cartItems.Select(item => new SessionLineItemOptions {
        PriceData = new SessionLineItemPriceDataOptions {
            Currency = "usd",
            ProductData = new SessionLineItemPriceDataProductDataOptions {
                Name = item.Game.Title
            },
            UnitAmount = (long)(effectivePrice * 100)
        },
        Quantity = 1
    }).ToList(),
    Mode = "payment",
    SuccessUrl = $"{domain}/cart/success?session_id={{CHECKOUT_SESSION_ID}}",
    CancelUrl = $"{domain}/cart"
};
var session = await new SessionService().CreateAsync(options);
```

2.4 SignalR Real-Time Notifications

Real-time notifications are delivered via a SignalR hub at the `/hub/notifications` endpoint. The `NotificationHub` (in `GameStore.PL/Hubs/`) manages connections via a `ConnectionTracker` singleton that maps `userId` to active connection IDs. When a notification is created (e.g., new support ticket, sale approval), the `NotificationService` persists the record to the `UserNotifications` table and broadcasts it to the recipient's active connections if online. The `NotificationsController` provides REST endpoints for fetching unread counts and marking notifications as read.

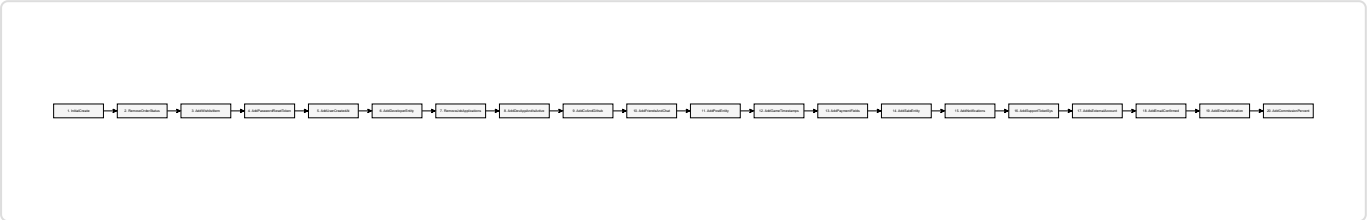
3. Migration History

The database schema evolved across 20 migrations throughout the development lifecycle (2026-05-03 to 2026-07-06). The table below chronicles each migration with its name, date, and purpose.

#	Migration Name	Date	Description
1	InitialCreate	2026-05-03	Base tables: Users, Games, Categories, GameCategories, Carts, CartItems, Orders, OrderItems, Libraries, LibraryGames, Reviews
2	RemoveOrderStatus	2026-06-23	Removed Status column from Orders table (replaced by PaymentStatus later)
3	AddWishlistItem	2026-06-23	WishlistItems table linking Users to Games with unique constraint on UserId+GameId
4	AddPasswordResetToken	2026-06-23	PasswordResetTokens table with Token, ExpiresAt, IsUsed columns
5	AddUserCreatedAt	2026-06-23	Added CreatedAt timestamp column to Users table
6	AddDeveloperEntity	2026-06-27	Developers table with Name, Slug, Description, Website, LogoUrl, Country; added DeveloperId FK to Games
7	RemoveJobApplications	2026-06-27	Dropped legacy JobApplications table
8	AddDeveloperApplicationAndIsActive	2026-06-27	DeveloperApplications table for approval workflow; IsActive flag on Developers

#	Migration Name	Date	Description
9	AddCvAndGithubToApplication	2026-06-27	Added CvFilePath and GithubUrl columns to DeveloperApplications
10	AddFriendsAndChat	2026-06-27	Friendships table (Requester, Receiver, Status) + Messages table with read receipts
11	AddPostEntity	2026-06-28	Posts table for user profile wall posts with 1000-char content limit
12	AddGameTimestamps	2026-06-28	Added CreatedAt and UpdatedAt columns to Games table
13	AddPaymentFields	2026-06-28	Added PaymentStatus, StripePaymentIntentId, StripeSessionId to Orders
14	AddSaleEntity	2026-06-29	Sales table for discount campaigns (NewPrice, StartDate, EndDate, Status, approval workflow)
15	AddUserNotifications	2026-06-29	UserNotifications table with Type, Category, ReferenceId for in-app notifications
16	AddSupportTicketSystem	2026-06-29	SupportTickets + SupportTicketReplies tables replacing old Reports table
17	AddIsExternalAccount	2026-07-01	Boolean flag on Users for Google OAuth / external account detection
18	AddEmailConfirmed	2026-07-02	EmailConfirmed flag on Users; existing users set to confirmed via SQL
19	AddEmailVerificationTokenTable	2026-07-02	EmailVerificationTokens table with Token, ExpiresAt, IsUsed; unique token index

#	Migration Name	Date	Description
20	AddCommissionPercent	2026-07-06	CommissionPercent (decimal 5,2) on OrderItems for per-order platform commission tracking



4. Technology Stack

Technology	Version	Purpose
.NET	8.0	Runtime and base class libraries
ASP.NET Core	8.0	MVC framework, middleware pipeline, and Kestrel web server
Entity Framework Core	9.0.8	ORM with Fluent API configuration and migrations
Microsoft.AspNetCore.Authentication.Google	8.0.10	Google OAuth 2.0 authentication provider
AutoMapper	12.0.1	Entity-to-ViewModel mapping
Stripe.net	52.1.0	Payment processing via Checkout Sessions and webhooks
SendGrid	9.29.3	Transactional email delivery (verification, password reset)

Technology	Version	Purpose
SignalR	Built-in	Real-time WebSocket notifications at /hub/notifications
BCrypt.Net-Next	4.0.3	Password hashing and verification
StackExchange.Redis	10.0.9	Distributed session caching (optional, falls back to in-memory)
SQL Server	Docker	Primary relational database (via EF Core SQL Server provider)
xUnit	2.9.3	Unit testing framework
Moq	4.20.72	Mocking framework for test isolation
FluentAssertions	6.12.2	Readable test assertions
Microsoft.NET.Test.Sdk	17.14.1	Test runner infrastructure
Bootstrap	5.1.0	Frontend CSS framework (responsive layout)
jQuery	3.6.0	Client-side DOM manipulation and AJAX
Font Awesome	6.x	Icon library for UI elements
Response Compression	Built-in	Brotli + Gzip compression (60-80% bandwidth reduction)
Rate Limiting (System.Threading.RateLimiting)	10.0.9	Brute-force protection on auth endpoints (login/register/resend)
Output Caching	Built-in	10-minute default cache TTL for read-heavy pages

5. Code Architecture Patterns

5.1 Repository Pattern

Data access is abstracted behind the generic `IRepository<T>` interface (in `GameStore.DAL/Repo/IRepository.cs`) implemented by `EfRepository<T>`. Each entity has standard CRUD operations via the generic repository, while the `GameStoreDbContext` exposes 21 `DbSet<T>` properties for direct LINQ queries when needed. Domain-specific services compose repository calls within their methods.

5.2 Unit of Work

The `IUnitOfWork` interface (in `GameStore.DAL/Repo/IUnitOfWork.cs`) wraps the `GameStoreDbContext` and provides a `Repository<T>()` method to access the generic repository for any entity. It exposes `SaveChangesAsync()` to commit all pending changes atomically. Services inject `IUnitOfWork` directly, ensuring that operations spanning multiple tables (e.g., creating an order, adding items to the library, and clearing the cart) commit as a single transaction or roll back entirely on failure.

5.3 Dependency Injection

The platform leverages ASP.NET Core's built-in DI container. Services are registered in `Program.cs` with `Scoped` lifetime (one instance per HTTP request). The registration follows a consistent pattern: each service interface maps to its implementation. Infrastructure services like `IJsonFileStore` use `Singleton` lifetime (stateless JSON file access). Background services like `TokenCleanupService` use `AddHostedService`. Constructor injection is used throughout all controllers and services.

```
// Program.cs - Service registration (actual DI registrations)
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddScoped<IGameService, GameService>();
builder.Services.AddScoped<IUserService, UserService>();
builder.Services.AddScoped<ICategoryService, CategoryService>();
builder.Services.AddScoped<IOrderService, OrderService>();
builder.Services.AddScoped<IOrderAnalyticsService, OrderAnalyticsService>();
```

```

builder.Services.AddScoped<IReviewService, ReviewService>();
builder.Services.AddScoped<ICartService, CartService>();
builder.Services.AddScoped<ILibraryService, LibraryService>();
builder.Services.AddScoped<IWishlistService, WishlistService>();
builder.Services.AddScoped<IDeveloperService, DeveloperService>();
builder.Services.AddScoped<IDeveloperApplicationService, DeveloperApplicationService>();
builder.Services.AddScoped<ISaleService, SaleService>();
builder.Services.AddScoped<ISupportTicketService, SupportTicketService>();
builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();
builder.Services.AddSingleton<IJsonFileStore, JsonFileStore>();
builder.Services.AddScoped<INotificationService, NotificationService>();
builder.Services.AddScoped<IChatService, ChatService>();
builder.Services.AddScoped<IFriendService, FriendService>();
builder.Services.AddScoped<IPostService, PostService>();
builder.Services.AddHostedService<TokenCleanupService>();

```

6. Service Layer Details

The BLL layer defines 19 service interfaces, each with a concrete implementation:

Interface	Key Methods
IAuthService	LoginAsync, RegisterAsync, ChangePasswordAsync, ConfirmEmailAsync, GetOrCreateExternalUserAsync, CreateVerificationTokenAsync, ConsumeVerificationTokenAsync
IUserService	GetAllAsync, GetByIdAsync, ChangeRoleAsync, DeleteAsync, GetTotalUsersAsync, GetUsersByRoleAsync, GetUserByUsernameAsync, SearchUsersAsync, UpdateProfileAsync
IGameService	GetAllWithCategoriesAsync, GetPagedAsync, GetByIdAsync, CreateAsync, UpdateAsync, DeleteAsync, GetTotalGamesAsync, GetFreeGamesCountAsync, GetGamesByCategoryAsync
ICategoryService	GetAllAsync, GetAllWithGameCountAsync, CreateAsync, UpdateAsync, DeleteAsync

Interface	Key Methods
ICartService	GetCartItemsAsync, AddToCartAsync, RemoveFromCartAsync
IOrderService	GetAllWithDetailsAsync, GetOrdersByUserAsync, GetOrderByByIdAsync, CompleteCheckoutAsync, CompleteFreeCheckoutAsync, GetByStripeSessionIdAsync, SetPaymentStatusAsync
IOrderAnalyticsService	GetTotalOrdersAsync, GetTotalRevenueAsync, GetAverageOrderValueAsync, GetRevenueByMonthAsync, GetOrdersByDayAsync, GetTopSellingGamesAsync, GetRevenueByCategoryAsync
IReviewService	GetAllWithDetailsAsync, DeleteAsync, GetByGameAsync, GetByDeveloperAsync, GetByUserAsync, CreateAsync
IWishlistService	GetWishlistAsync, IsInWishlistAsync, AddToWishlistAsync, RemoveFromWishlistAsync
ILibraryService	GetLibraryGamesAsync, HasGame, AddGameToLibraryAsync, RemoveGameFromLibraryAsync
IChatService	SendMessageAsync, GetConversationAsync, GetUnreadCountAsync, MarkAsReadAsync, GetConversationsAsync
ISupportTicketService	CreateAsync, GetByIdAsync, GetUserTicketsAsync, GetAllAsync, AddReplyAsync, UpdateStatusAsync, IsOwnerAsync
ISaleService	GetByIdAsync, GetPendingAsync, GetByDeveloperAsync, CreateAsync, ApproveAsync, RejectAsync
IDeveloperService	GetByUserIdAsync, GetByIdAsync, GetAllAsync, GetGamesAsync, CreateOrUpdateProfileAsync, GetDashboardStatsAsync, GetGameStatsAsync, DeleteAsync, DemoteAsync, ReactivateAsync
IDeveloperApplicationService	GetByIdAsync, GetByUserIdAsync, GetAllAsync, SubmitAsync, ApproveAsync, RejectAsync

Interface	Key Methods
IFriendService	GetFriendsAsync, GetPendingRequestsAsync, GetFriendIdsAsync, SendRequestAsync, AcceptRequestAsync, RejectRequestAsync, RemoveFriendAsync
IPostService	CreateAsync, GetUserPostsAsync, DeleteAsync, GetUserPostCountAsync, GetLastPostTimeAsync
IGameFileService	File upload/delete game binary files, screenshots, cover images
IGameAccessService	Verifies user ownership/download access rights for game files

7. Version Control & Branching Strategy

The project is hosted on **GitHub** with a GitFlow-inspired branching model. The CI pipeline (defined in `.github/workflows/ci.yml`) triggers on pushes to `main` and `develop`, and on pull requests targeting `main`.

Branch	Purpose	Protection	Deployment Target
<code>main</code>	Production-ready code. Only merged via PR from <code>develop</code> or <code>hotfix/*</code> .	Requires CI pass + code review	Production environment
<code>develop</code>	Integration branch for completed features. All <code>feature/*</code> branches merge here.	Requires CI pass	Staging environment
<code>feature/*</code>	Individual feature work (e.g., <code>feature/stripe-webhook</code> , <code>feature/signalr-chat</code>).	None	Developer local
<code>hotfix/*</code>	Urgent production fixes branched from <code>main</code> , merged back to both <code>main</code> and <code>develop</code> .	Requires CI pass	Production environment

Commit Convention: <type>(<scope>): <description> — e.g., feat(auth): add Google OAuth callback, fix(cart): resolve race condition in checkout, test(order): add CompleteCheckout unit tests.

8. CI/CD Pipeline

The continuous integration pipeline runs on **GitHub Actions** (ubuntu-latest) with SQL Server 2022 and Redis 7-alpine as service containers. The pipeline executes on every push to main/develop and on every PR to main.

Stage	Command	Description
1. Checkout	actions/checkout@v4	Clone repository with full Git history
2. Setup .NET	actions/setup-dotnet@v4 (8.0.x)	Install .NET 8 SDK
3. Restore	dotnet restore	Restore NuGet packages for the solution
4. Build	dotnet build -c Release --no-restore	Compile all four projects in Release mode
5. Test	dotnet test -c Release --no-restore --verbosity normal	Run 185 xUnit tests against SQL Server + Redis service containers
6. Publish	dotnet publish -c Release -o publish	Produce deployable artifacts
7. Upload Artifact	actions/upload-artifact@v4	Store published output as gamestore-app artifact

Service Containers: SQL Server 2022 (port 1433, health check with sqlcmd) and Redis 7-alpine (port 6379, health check with redis-cli ping) are provisioned as Docker service containers for the test stage, ensuring integration tests exercise the real database and cache.

9. README & Deployment Guide

The project includes a comprehensive **1098-line README** at `GameStore.PL123/README.md` covering the following sections:

Quick Start (Local Development)

```
# 1. Clone the repository
git clone https://github.com/your-org/gamestore.git
cd gamestore/GameStore.PL123

# 2. Configure secrets (Stripe, SendGrid, Google OAuth)
dotnet user-secrets init
dotnet user-secrets set "Stripe:SecretKey" "sk_test_..."
dotnet user-secrets set "Stripe:PublishableKey" "pk_test_..."
dotnet user-secrets set "SendGrid:ApiKey" "SG...."
dotnet user-secrets set "Google:ClientId" "..."
dotnet user-secrets set "Google:ClientSecret" "..."

# 3. Update connection string in appsettings.json (default: LocalDB)

# 4. Run the application
dotnet run --project Game-Store
# Opens at https://localhost:5001

# 5. Run tests
dotnet test GameStore.Tests
```

Docker Deployment

```
# From repository root
docker compose up -d
# App: http://localhost:5000
# SQL Server: localhost:1433
# Redis: localhost:6379
```

System Requirements

Environment	Requirement
Development	.NET 8 SDK, SQL Server LocalDB or Express, Visual Studio 2022 or VS Code
Docker	Docker Desktop 4.x+, 4 GB RAM allocated, 10 GB disk
Production	Windows Server / Linux container, .NET 8 Runtime, SQL Server 2022+, Redis 7+, reverse proxy (nginx/IIS)
Browser	Chrome 120+, Edge 120+, Firefox 120+ (for SignalR WebSocket transport)

Configuration

All sensitive configuration is managed via **.NET User Secrets** in development and **environment variables** in production. The `appsettings.json` file contains only non-sensitive defaults. Key configuration sections:

- `ConnectionStrings:GameStoreConnection` — SQL Server connection string
- `ConnectionStrings:Redis` — Redis connection string (empty = in-memory cache fallback)
- `Stripe:SecretKey` / `Stripe:PublishableKey` / `Stripe:WebhookSecret` — Stripe API credentials
- `SendGrid:ApiKey` / `SendGrid:FromEmail` / `SendGrid:FromName` — Email provider
- `Google:ClientId` / `Google:ClientSecret` — OAuth 2.0 credentials
- `PasswordSalt` — Salt for legacy SHA256 password hashing